

Feature-Guided Domain Selection for Fast Fractal Image Compression of High-Resolution Images

Zhen-Rong Wu
Dept. of Computer Science
and Information Engineering
National Taiwan Normal University
41247012S@ntnu.edu.tw

Darrin Lin
Dept. of Computer Science
and Information Engineering
National Taiwan Normal University
41247022S@ntnu.edu.tw

Liang-Yun Ko
Dept. of Computer Science
and Information Engineering
National Taiwan Normal University
41247066S@ntnu.edu.tw

Abstract—Fractal Image Compression (FIC) attains resolution-independent decoding by exploiting block self-affine redundancy, but its encoder must search an enormous domain pool, which becomes prohibitive at high resolution. A well-known line of work replaces the exhaustive search with metaheuristics such as Particle Swarm Optimization (PSO). We revisit this strategy on 1024×1024 images and find that it has a structural flaw: at this scale a single range block faces over half a million domain-isometry candidates, so a swarm of a few dozen particles covers a vanishing fraction of the space and converges far from the optimum. Through a sequence of controlled experiments—PSO, memetic PSO (PSO with local search), and a feature-guided memetic PSO—we show that the bottleneck is not the optimizer but the *quality of the search space* it is given. This motivates Feature-Guided Domain Selection (FGDS), in which the heuristic is moved out of per-block matching and into *domain-block selection*: an 8-dimensional isometry-invariant descriptor places every block in a feature space, a KD-Tree returns the K most similar domains per range block, and a small exhaustive search within that pool finds the optimal mapping. An ablation shows that once the pool is reduced to $K = 80$ candidates, exhaustive search is empirically superior than running PSO inside it—so the optimizer can be removed entirely. Replacing pairwise candidate selection with a KD-Tree cuts pool construction from ~ 180 s to under 0.5 s ($\sim 1390\times$), and a Genetic Algorithm trained once on a single held-out image learns how much each feature dimension should count, yielding a small but consistent additional gain that transfers to every test image. On ten DIV2K images, FGDS improves PSNR by 1.7–3.2 dB over PSO and 0.5 dB over feature-guided PSO while running $\sim 4\times$ faster, with all comparisons significant under the Wilcoxon signed-rank test ($p < 0.01$, $n = 10$). A random-pool control isolates the gain to the feature-guided selection itself rather than to mere search-space reduction (+3.25 dB over a same-size random pool).

Index Terms—Fractal image compression, feature-guided search, domain block selection, KD-Tree, genetic algorithm, isometry-invariant features, metaheuristics

I. INTRODUCTION

Fractal Image Compression (FIC) encodes an image as a set of contractive affine maps between image regions, exploiting self-affine redundancy to achieve compact, resolution-independent representations [1], [2]. Decoding is fast: iterating the stored maps from any starting image converges to a fixed-point reconstruction. Encoding, however, is the opposite. For every non-overlapping range block, the encoder must find, among all domain blocks and all eight geometric isometries, the transformation that minimizes reconstruction error. This

search grows quadratically with image resolution and is the dominant cost of FIC.

A long-standing line of work replaces the exhaustive search with metaheuristics. Muruganandham and Wahida Banu [3] applied Particle Swarm Optimization (PSO) to reduce encoding time, and genetic algorithms [4]–[6] and more recent swarm variants [7] have been explored in the same spirit. These methods treat the domain index and isometry of each range block as the variables to be optimized, and search that space with a population of candidate solutions.

We began by reproducing this strategy on high-resolution (1024×1024) images and quickly hit a wall. At this scale a single range block has $P \times 8 \approx 5.2 \times 10^5$ domain-isometry candidates. A PSO swarm of 40 particles therefore inspects well under 0.1% of the space at initialization, and because the FIC objective over domain indices is highly non-smooth—spatially adjacent domains can have completely different content—the swarm has no usable gradient to follow and converges to solutions far from the optimum. Adding memetic local search [8] improves quality but cannot repair this: if the swarm settles in the wrong region of the pool, refining its neighborhood does not help.

This led us to a different diagnosis. The difficulty is not that the optimizer is weak; it is that *the search space handed to the optimizer is bad*. All domain-matching metaheuristics search blindly over the full pool, spending most of their budget on structurally dissimilar blocks that can never produce a good match. Classical FIC acceleration recognized a version of this—Fisher’s block classification [9] and Saupe’s reformulation of block matching as nearest-neighbor search in feature space [10]—but these ideas predate, and were never reconciled with, the metaheuristic FIC literature for high-resolution images.

Our central decision is therefore to **move the heuristic from block matching to block selection**. Instead of optimizing which domain matches each range block, we use a fast, structure-aware heuristic to *select* a small, high-quality candidate pool of domains per range block, and then search that pool to optimality. We call this **Feature-Guided Domain Selection (FGDS)**. Concretely, FGDS (i) computes an 8-dimensional isometry-invariant descriptor for every block, (ii) uses a KD-Tree to retrieve the K nearest domains for each

range block in a learned weighted feature space, and (iii) exhaustively evaluates the $K \times 8$ candidates in that pool.

This paper presents the full reasoning trail that produced FGDS, because each step is itself a result. We show experimentally that (a) memetic local search helps PSO but costs time; (b) restricting PSO to a feature-guided pool helps more; (c) once the pool is small, *exhaustive search beats PSO inside it*, so the optimizer can be dropped; (d) the expensive part that remains—pairwise candidate selection—can be replaced by a KD-Tree at no quality cost; (e) a candidate pool of $K=80$ is a sweet spot; (f) a Genetic Algorithm can learn, offline and once, how important each feature dimension is, and these weights transfer across images; and (g) a same-size *random* pool is far worse than the feature-guided pool, proving that the gain comes from the selection, not from search-space reduction alone.

The contributions are:

- 1) **A reframing of metaheuristics in FIC** from per-block domain matching to domain-block *selection*, supported by an ablation showing that exhaustive search dominates PSO inside a feature-guided pool.
- 2) **FGDS**, an 8-dimensional isometry-invariant descriptor plus KD-Tree nearest-neighbor selection, which cuts candidate-pool construction from ~ 180 s to under 0.5 s ($\sim 1390\times$) and reduces the per-range search space by over 6×10^3 .
- 3) **A GA-based feature-weight learner** that discovers, offline on a single held-out image, the relative importance of each feature dimension and transfers it to unseen images, motivated by the structure of the FIC affine model.
- 4) **A controlled study on ten high-resolution images**, including a random-pool control and a K -sensitivity sweep, with Wilcoxon signed-rank significance ($n=10$), establishing that the feature-guided pool—not the inner optimizer—is the source of the improvement.

II. BACKGROUND AND RELATED WORK

A. Fractal Image Compression

FIC originates from the theory of Iterated Function Systems (IFS) [1], [11], whose Collage Theorem and Contractive Mapping Fixed-Point Theorem justify representing an image by contractive self-maps [12]. Jacquin [2], [13] turned this into a practical block-based coder.

A gray image of size $N \times N$ is partitioned into a range pool $\mathcal{R}$ of $M = (N/L)^2$ non-overlapping $L \times L$ range blocks and a domain pool $\mathcal{D}$ of $P = \lfloor (N-2L)/\delta + 1 \rfloor^2$ overlapping $2L \times 2L$ domain blocks taken with stride δ ; each domain block is downsampled to $L \times L$ by 2×2 average pooling. For range block v and a domain block u , the encoder seeks a contractive affine map combining one of eight Dihedral isometries T_k ($k \in \{0, \dots, 7\}$) with a contrast s and brightness o :

$$\hat{v} = s \cdot T_k(\text{downsample}(u)) + o. \quad (1)$$

For a fixed (u, k) the optimal s, o minimize the Mean Squared Error

$$\text{MSE}(v, \hat{v}) = \frac{1}{L^2} \sum_{i,j} (v(i, j) - \hat{v}(i, j))^2 \quad (2)$$

and have the closed-form least-squares solution

$$s = \frac{L^2 \sum v u_k - (\sum v)(\sum u_k)}{L^2 \sum u_k^2 - (\sum u_k)^2}, \quad o = \frac{\sum v - s \sum u_k}{L^2}, \quad (3)$$

where $u_k = T_k(\text{downsample}(u))$. The best isometry is $k^* = \arg \min_k \text{MSE}(v, s_k u_k + o_k)$, and contractivity $|s| < 1$ is enforced for decode convergence. We refer to one evaluation of (2)–(3) for a single (u, k) pair as a *Fitness Function Evaluation (FFE)*; it is the unit of search cost shared by all methods in this paper.

Quality is reported as Peak Signal-to-Noise Ratio, $\text{PSNR} = 10 \log_{10}(255^2 / \text{MSE}_{\text{avg}})$, with MSE_{avg} the mean over range blocks of the decoded image.

Compression ratio: Each code stores a domain index, k^* , and quantized s, o , costing $b_{\text{code}} = \lceil \log_2 P \rceil + 3 + b_s + b_o$ bits. With $N=1024$, $L=4$, $\delta=4$ we have $P=65,025$, $b_s=b_o=8$, hence $b_{\text{code}}=35$ and a compression ratio $\text{CR} = N^2 \cdot 8 / (M b_{\text{code}}) \approx 3.7:1$. Because all methods use identical partitioning and quantization, CR is the same throughout; the comparison is purely about achievable PSNR and time at a fixed CR.

B. Acceleration and metaheuristic FIC

Exhaustive FIC evaluates all $P \times 8$ candidates per range block. Classical accelerations include quad-tree partitioning [9], tree-structured codebook classification [14], [15], and parallel hardware [16], [17]. Saupe [10] reframed block matching as multi-dimensional nearest-neighbor search—conceptually the closest ancestor of our approach. Independently, the metaheuristic line treats (d, k) as decision variables and searches with PSO [3], [18], [19], genetic algorithms [4]–[6], or pyramid swarm strategies [7]. Our work connects the two: we use feature-space nearest-neighbor *selection* to build the search space, then argue and demonstrate that the metaheuristic *matcher* inside it is unnecessary, and we additionally use a metaheuristic (GA) one level up, to *design* the feature space.

III. FROM PSO TO FEATURE-GUIDED SELECTION

This section describes the methods we compare, in the order in which they led us to FGDS. All share the fitness function of (2)–(3) and the same FFE accounting, so differences reflect only *search strategy*.

A. Standard PSO (baseline)

Following [3], each particle is a continuous vector $\mathbf{X}_j = [d_j, k_j]$ decoded to a (domain index, isometry) pair by rounding and clamping. The swarm updates

$$\mathbf{V}_j(t+1) = w \mathbf{V}_j(t) + c_1 r_1 (\mathbf{P}_{\text{best},j} - \mathbf{X}_j) + c_2 r_2 (\mathbf{G}_{\text{best}} - \mathbf{X}_j), \quad (4)$$

$$\mathbf{X}_j(t+1) = \mathbf{X}_j(t) + \mathbf{V}_j(t+1), \quad (5)$$

with $N_p=40$, $w=0.9$, $c_1=c_2=2.0$, velocity clamped to $\pm v_{\text{max}}$. The search domain is $[0, P-1] \times [0, 7]$.

B. Memetic PSO

Memetic PSO augments the swarm with periodic local search on the top 20% of particles. **Isometry-LS** tries the 7 alternative isometries of a particle's current domain (≤ 7 FFEs); **Spatial-LS** tries the 8 spatially adjacent domain blocks with the same isometry (≤ 8 FFEs), exploiting spatial self-similarity. A final pass refines the global best. All LS evaluations count against the FFE budget. This is the first lesson in our trail: local search reliably raises PSNR but costs additional time (Sec. VI-A).

C. Feature-Guided PSO (FG-PSO)

The second lesson motivated restricting where PSO searches. FG-PSO first builds, for each range block, a feature-guided candidate pool of K domains (Sec. IV), then runs the same memetic PSO inside the reduced subspace $[0, K-1] \times [0, 7]$ rather than the full pool. Shrinking the space from $\sim 5 \times 10^5$ to $K \times 8$ candidates lets a small swarm cover a meaningful fraction of it, and FG-PSO indeed improves on both PSO variants. This is the method our earlier work proposed.

D. The ablation that removed the optimizer

FG-PSO raised an obvious question: if the pool already contains only $K=40$ domains (320 candidates), is PSO doing anything that exhaustive search could not? We therefore replaced the inner PSO with a plain exhaustive scan of all $K \times 8$ candidates—*feature-guided exhaustive search*, the seed of FGDS. As Sec. VI-A shows, this exhaustive variant matches or beats FG-PSO at higher PSNR, fewer FFEs, zero randomness, and lower time. Within a few-hundred-candidate space, exhaustive search is simply the better choice, and the metaheuristic matcher can be discarded. What remains valuable is the *selection* of the pool—which is where we direct the rest of the paper.

IV. PROPOSED METHOD: FEATURE-GUIDED DOMAIN SELECTION

FGDS replaces metaheuristic block matching with feature-guided block selection followed by a small exhaustive search. The final pipeline is

Block $\rightarrow$ Feature Extraction $\rightarrow$ Weighted Feature Space
 $\rightarrow$ KD-Tree NN $\rightarrow$ Candidate Pool $\rightarrow$ Exhaustive Search.

An offline Genetic Algorithm (Sec. IV-D) learns the feature weights once and is not part of per-image encoding.

A. Stage 1: Isometry-invariant features

The descriptor must (R1) discriminate structurally dissimilar blocks and (R2) be *isometry-invariant*, since a domain under any of the eight Dihedral transforms is the same candidate (the isometry is searched separately in Stage 3). For a block b of size $L \times L$ we use

$$\mathbf{f}(b) = [\mu, \sigma, \bar{g}_h, \bar{g}_v, \hat{q}_1, \hat{q}_2, \hat{q}_3, \hat{q}_4] \in \mathbb{R}^8. \quad (6)$$

Algorithm 1 FGDS Encoding Pipeline

Require: Image I ($N \times N$); range size L ; stride δ ; pool size K ; weights $\mathbf{w} \in \mathbb{R}^8$

Ensure: Fractal codes $\mathcal{C}$

```

1: // Stage 1: Feature Extraction
2:  $\mathbf{f}_R \leftarrow \text{EXTRACTFEATURES}(\text{range blocks}) \{(M, 8)\}$ 
3:  $\mathbf{f}_D \leftarrow \text{EXTRACTFEATURES}(\text{domain blocks}) \{(P, 8)\}$ 
4: z-score normalize  $\mathbf{f}_R, \mathbf{f}_D$  using  $(\mu_D, \sigma_D)$ 
5: Apply weights:  $\hat{\mathbf{f}} \leftarrow \hat{\mathbf{f}} \odot \mathbf{w}$  {element-wise}
6: // Stage 2: KD-Tree Candidate Pool
7: Build KD-Tree  $\mathcal{T}$  from  $\hat{\mathbf{f}}_D$ 
8: for each range block  $v_i$  do
9:    $\mathcal{P}_i \leftarrow \mathcal{T}.\text{query}(\hat{\mathbf{f}}_{R,i}, K)$  {top- $K$  neighbors}
10: end for
11: // Stage 3: Exhaustive Search in Reduced Space
12: for each range block  $v_i$  do
13:    $(d^*, k^*, s^*, o^*) \leftarrow \arg \min_{j \in \mathcal{P}_{i,k}} \text{MSE}(v_i, sT_k(u_j) + o)$ 
14:    $\mathcal{C}_i \leftarrow (d^*, k^*, s^*, o^*)$ 
15: end for
16: return  $\mathcal{C}$ 

```

Here μ and σ are the block mean and standard deviation (both exactly isometry-invariant). $\bar{g}_h = \text{mean}|b(i, j+1) - b(i, j)|$ and $\bar{g}_v = \text{mean}|b(i+1, j) - b(i, j)|$ are mean gradient magnitudes capturing texture; they are invariant under flips and merely swap under 90° rotation (quasi-invariant). The $\hat{q}_1 \leq \dots \leq \hat{q}_4$ are the four quadrant means *sorted ascending*: rotations and reflections only permute quadrants, so sorting makes them fully isometry-invariant. Each feature costs $O(L^2)$.

Features are z-score normalized with domain-pool statistics, $\hat{\mathbf{f}} = (\mathbf{f} - \mu_D)/(\sigma_D + 10^{-8})$, so every dimension contributes comparably to Euclidean distance before weighting.

B. Stage 2: KD-Tree candidate selection

For range block v_i we want the K domains nearest in weighted feature space,

$$\mathcal{P}_i = \arg \min_{|S|=K} \sum_{j \in S} \|\mathbf{w} \odot (\hat{\mathbf{f}}_{R,i} - \hat{\mathbf{f}}_{D,j})\|^2. \quad (7)$$

Pre-multiplying both feature sets by $\mathbf{w}$ turns this into plain Euclidean nearest-neighbor search, so a standard KD-Tree [20] applies. A naive pairwise scan costs $O(MPF)$ (here ~ 180 s); the KD-Tree costs $O(PF \log P)$ to build and $O(MK \log P)$ to query, completing in under 0.5 s in practice. With $K=80$ the per-range space shrinks from $P \times 8 = 520,200$ to 640 candidates, over 6×10^3 smaller. KD-Trees remain efficient at $F=8$ dimensions (the curse of dimensionality sets in well above ~ 20).

C. Stage 3: Exhaustive search in the pool

Within $\mathcal{P}_i$ we evaluate all $K \times 8$ domain-isometry pairs via (2)–(3) and keep the minimum-MSE candidate with $|s| < 1$. Unlike PSO, which rounds continuous positions to discrete indices and may skip candidates, this scan is exact within the pool. With $K=80$ it costs 640 FFEs per block—fewer than

Algorithm 2 GA-Based Feature-Weight Optimization

Require: Training image I ; population N_{pop} ; generations G ; sample ratio ρ

Ensure: Optimized weights $\mathbf{w}^*$

- 1: Precompute blocks, features $\mathbf{f}_R, \mathbf{f}_D$, isometries
- 2: Sample ρM range-block indices $\mathcal{S}$
- 3: $\mathbf{w}_0 \leftarrow \mathbf{1}$; $\mathbf{w}_{1..N_{\text{pop}}-1} \sim U(0.3, 2.5)^8$
- 4: **for** $g = 0$ to G **do**
- 5: Evaluate $f(\mathbf{w}_i)$ on $\mathcal{S}$ for all i
- 6: Copy top-2 elites to next generation
- 7: **for** each remaining slot **do**
- 8: parents $\leftarrow$ tournament selection (size 3)
- 9: child $\leftarrow$ BLX- α crossover ($\alpha=0.5$, rate 0.8) [22]
- 10: child $\leftarrow$ Gaussian mutation ($\sigma=0.25$, rate 0.4)
- 11: child $\leftarrow$ max(child, 0.05)
- 12: **end for**
- 13: **end for**
- 14: **return** best $\mathbf{w}$ over all generations

the budgets given to the PSO baselines—yet it is guaranteed optimal over the selected domains.

D. Learning feature weights with a Genetic Algorithm

Equal weighting ($\mathbf{w} = \mathbf{1}$) assumes all eight dimensions matter equally for pool quality, which is not true: the dimensions play different roles in the affine match of (3). We therefore learn $\mathbf{w} \in \mathbb{R}_{>0}^8$ with a Genetic Algorithm [21] run *once, offline*, on a single training image disjoint from the test set, and apply the result to all test images at zero extra encoding cost. This is a deliberately different use of a metaheuristic from the discarded PSO matcher: GA optimizes the *design of the feature space*, not the per-block search.

Fitness: An individual is a weight vector $\mathbf{w}$. To evaluate it we run weighted feature extraction $\rightarrow$ KD-Tree $\rightarrow$ exhaustive search on a random 2% sample of range blocks ($\approx 1,300$ blocks for 1024^2) and return the average MSE; fitness is $-\text{avgMSE}$. Raw features, domain statistics, and isometry tables are pre-computed once, so each evaluation only re-weights, rebuilds the tree, and scans the sample.

Operators: Individual 0 is the equal-weight baseline; the rest are drawn from $U(0.3, 2.5)^8$. Selection is tournament (size 3); crossover is BLX- α ($\alpha=0.5$, rate 0.8) [22]; mutation perturbs 1–3 dimensions by $\mathcal{N}(0, 0.25)$ at rate 0.4; weights are clamped positive; two elites are preserved. We use $N_{\text{pop}}=16$, $G=10$, $\rho=0.02$, $K=80$, training on the held-out image 0003. The sampled MSE improves from 72.82 (equal weights) to 71.77 over ten generations, and the search converges to

$$\mathbf{w}^* = [0.18, 1.36, 0.97, 0.96, 1.29, 1.04, 1.00, 0.75] \quad (8)$$

for $[\mu, \sigma, \bar{g}_h, \bar{g}_v, \hat{q}_1, \hat{q}_2, \hat{q}_3, \hat{q}_4]$. The learned vector is a striking confirmation of the model-structure argument below: the mean μ receives by far the smallest weight (0.18) and the standard deviation σ the largest (1.36)—a ratio of $7.6\times$ —exactly the “down-weight μ , up-weight σ ” prediction. The gradient and

TABLE I: Per-range search cost and preprocessing.

Method	Per-range cost	Preprocessing
Full search	$O(P \cdot 8)$	$O(PL^2)$
PSO (budget B)	$O(B)$	$O(PL^2)$
FGDS (ours)	$O(K \cdot 8)$	$O(PF \log P + MK \log P)$

quadrant features stay near 1.0, consistent with their secondary role at $L=4$.

Why one training image generalizes: The weights capture structure of the FIC affine model rather than image content, so they transfer. From (3), $s = \text{Cov}(v, u_k) / \text{Var}(u_k)$ and $o = \bar{v} - s\bar{u}_k$. Consider a mismatch in each feature between a range block and a candidate domain: *Mean* (μ). Any difference $\bar{v} - \bar{u}_k$ is absorbed exactly by the additive offset o . A μ -mismatch in the pool barely affects final MSE, so μ should be *down-weighted* for selection. *Standard deviation* (σ). Since $s \approx \sigma_v / \sigma_{u_k}$ for well-correlated blocks, a large σ -mismatch either forces $|s| \geq 1$ (the candidate is discarded by contractivity) or leaves a residual the affine model cannot remove. This is a *multiplicative* penalty, so σ should be *up-weighted*. *Gradient/quadrant features.* These encode texture and spatial layout; their usefulness depends on the block size L (coarser at small L). The key point is that “ σ drives the multiplicative contrast, μ is free” follows from the *form* of (3), not from any image. Hence the optimal weight ratio is approximately image-independent, and a GA trained on one image transfers. The learned weights of (8) bear this out directly— μ is suppressed and σ amplified by the predicted ordering. Empirically (Sec. VI-B) these weights improve PSNR on every one of the ten unseen test images, never degrading any, confirming the argument. Because the per-image gain is small, the main value of the GA is as evidence that *which features matter* can be learned automatically—a transferable principle beyond FIC—rather than as a large quality boost.

E. Computational complexity

Table I summarizes the costs. FGDS trades a sub-second KD-Tree preprocessing step for a per-range cost of only $K \times 8$ FFEs. A full search at 1024^2 would require $MP \times 8 \approx 3.4 \times 10^{10}$ FFEs per image; FGDS at $K=80$ uses $M \times 640 \approx 4.2 \times 10^7$, a reduction of over $800\times$, while the GA cost is one-time and offline.

V. EXPERIMENTAL SETUP

1) *Hardware and implementation:* Experiments run on an AMD EPYC 7313P 16-core server with FreeBSD 15.0. All methods are implemented in Python/NumPy; FGDS uses SciPy’s KDTree.

2) *Data:* We use ten images from DIV2K [23]—0002, 0007, 0029, 0030, 0032, 0037, 0043, 0049, 0051, 0060—each resized to 1024×1024 and converted to 8-bit grayscale. The GA training image (0003) is *not* in the test set, so all reported GA results are on unseen images.

3) *FIC parameters*: $L=4$, $2L=8$, $\delta=4$, giving $M=65,536$ range and $P=65,025$ domain blocks, $CR \approx 3.7:1$, decode 20 iterations.

4) *Budget and protocol*: We fix an FFE budget of 800 per range block. This is generous for FGDS—at $K=80$ it finishes in $K \times 8 = 640$ FFEs and is flat thereafter—and is chosen so the convergence plots show, on a common axis, exactly when each method saturates. Stochastic methods (PSO, memetic PSO, FG-PSO) run 5 times per image with seeds $\{42, \dots, 46\}$; FGDS is deterministic. Pairwise comparisons use the Wilcoxon signed-rank test [24] over the $n=10$ images.

5) *Experiments*: The study is a single connected line of evidence, organized as four runs: **(E1)** PSO vs. memetic PSO vs. FG-PSO vs. FGDS-exhaustive, all feature-based methods at $K=40$, establishing the optimizer-removal result; **(E2)** the three FGDS variants—pairwise, KD-Tree, and KD-Tree+GA weights—at $K=80$, isolating the preprocessing speedup and the GA contribution; **(E3)** a K -sensitivity sweep $K \in \{5, 10, 20, 40, 80, 160\}$ for KD-Tree FGDS, locating the sweet spot; **(E4)** a same-size random-pool control vs. FGDS, both at $K=80$, attributing the gain to feature-guided selection. E1 fixes $K=40$ because that was the pool size of our original FG-PSO; the K -sweep of E3 then justifies adopting $K=80$ for E2 and E4.

VI. RESULTS

A. E1: Optimizer vs. search-space quality

Table II traces the full trail. Memetic PSO beats plain PSO on every image (+0.28 dB on average) but the local search adds cost relative to the per-iteration work—confirming that LS helps quality. FG-PSO then improves substantially over memetic PSO (+1.12 dB) by giving the swarm a feature-guided pool to search, confirming that *search-space quality*, not optimizer tuning, is what was missing. Finally, replacing the inner PSO with exhaustive search over the same $K=40$ pool—FGDS—adds another +0.53 dB while using fewer FFEs (320 vs. the 800 budget), no randomness, and far less time.

Magnitude. FGDS exceeds PSO by 1.26–3.22 dB (mean +1.93), memetic PSO by 1.13–2.72 dB (mean +1.65), and FG-PSO by 0.42–0.69 dB (mean +0.53). In lossy compression even 0.5 dB is meaningful.

Speed. FGDS averages 358 s per image including preprocessing, versus 1420 s (PSO), 1317 s (memetic PSO), and 1269 s (FG-PSO)—a 3.5–4.0 $\times$ speedup. The savings come from the tiny per-range search (320 vs. hundreds of FFEs spread over swarm bookkeeping) and from determinism (no repeated runs).

The optimizer is unnecessary. The decisive ablation is FG-PSO vs. FGDS: same pool, with vs. without the inner PSO. FGDS wins on all ten images at fewer evaluations. Within a few-hundred-candidate space, a complete scan dominates any stochastic searcher, so the metaheuristic matcher can be dropped. This is what justifies removing PSO from the final method.

Fig. 1 makes the trail visible. Even at FFE = 0, the feature-guided methods (FG-PSO, FGDS) begin 2–3 dB above the

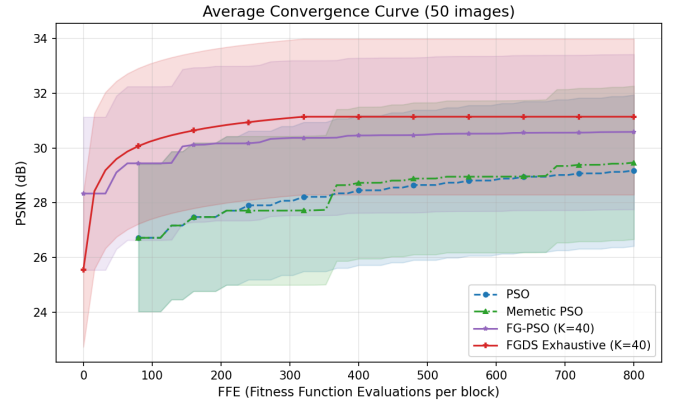


Fig. 1: E1 convergence averaged over the ten images (5 runs each; $\pm 1\sigma$ bands). The feature-guided methods start far above the PSO baselines; FGDS saturates at 320 FFEs and remains the highest.

PSO baselines, because a random draw from the feature pool is already a good match. FGDS climbs fastest and saturates at $K \times 8 = 320$ FFEs, after which its curve is flat—all pool candidates have been evaluated—while the PSO baselines are still slowly improving at the 800-FFE budget and never close the gap.

B. E2: KD-Tree acceleration and GA weights

Two questions are answered in Table III.

Does the KD-Tree change results? No—and that is the point. The KD-Tree returns the same nearest neighbors as the pairwise scan, so PSNR and MSE are *identical* to four significant figures (31.21 dB, 56.27 MSE). What changes is candidate-selection time: from 179.6 s (pairwise) to 0.13 s (KD-Tree), a $\sim 1390\times$ reduction, turning the dominant preprocessing cost into a negligible one. Feature extraction itself is unchanged at ~ 7.2 s.

What does the GA add? Learned weights raise PSNR on all ten images, by +0.01 to +0.09 dB (mean +0.05 dB, 31.21 $\rightarrow$ 31.26), with a matching MSE drop (56.27 $\rightarrow$ 55.60) and no extra encoding cost. The improvement is small but uniformly positive—never negative on any image (Wilcoxon $W^+ = 55$, $p < 0.01$, $n = 10$)—which is exactly the signature predicted by the model-structure argument of Sec. IV-D: the GA, trained only on the held-out image 0003, learns the weight vector of (8)—down-weighting the offset-absorbed mean ($w_\mu = 0.18$) and up-weighting the contrast-driving standard deviation ($w_\sigma = 1.36$)—a property of the affine model that holds for every image. We report this honestly as a modest, reliable gain whose chief significance is methodological: it shows the importance of each feature can be *learned* rather than hand-set.

Fig. 2 shows the three variants on image 0030; the curves are visually indistinguishable, reflecting the near-identical PSNR, with the GA curve marginally on top.

TABLE II: E1: PSO baselines vs. FGDS (feature-guided exhaustive, $K=40$) on ten 1024×1024 images. Stochastic methods: mean over 5 runs (PSNR std ≤ 0.03 dB). FGDS is deterministic. Best per image in bold.

Image	PSO		Memetic PSO		FG-PSO ($K=40$)		FGDS ($K=40$)	
	Time(s)	PSNR	Time(s)	PSNR	Time(s)	PSNR	Time(s)	PSNR
0002	1432.1	28.73	1299.3	28.87	1286.4	29.58	364.5	30.00
0007	1429.8	26.32	1301.7	26.45	1289.1	27.14	363.8	27.58
0029	1441.3	23.47	1301.2	23.69	1285.3	24.89	365.6	25.43
0030	1421.3	30.52	1302.2	30.84	1284.8	31.71	358.5	32.26
0032	1412.5	33.78	1308.4	34.03	1252.5	35.00	354.4	35.47
0037	1414.3	32.17	1345.8	32.65	1250.6	34.07	354.5	34.66
0043	1412.0	27.70	1343.8	28.20	1249.8	30.26	354.3	30.92
0049	1412.9	29.96	1337.2	30.12	1249.7	31.23	354.8	31.71
0051	1411.1	27.78	1302.2	28.04	1270.9	28.86	354.9	29.31
0060	1410.7	28.31	1330.0	28.60	1268.7	29.99	354.8	30.68
Avg	1419.8	28.87	1317.2	29.15	1268.8	30.27	358.0	30.80

TABLE III: E2: three FGDS variants at $K=80$, averaged over ten images. Times in seconds; CandSel = candidate selection. PSNR/MSE for pairwise and KD-Tree are identical by construction.

Variant	PSNR	MSE	FeatExt	CandSel	Encode
Pairwise	31.21	56.27	7.20	179.57	697.80
KD-Tree	31.21	56.27	7.22	0.13	692.81
KD-Tree + GA	31.26	55.60	7.22	0.12	692.10

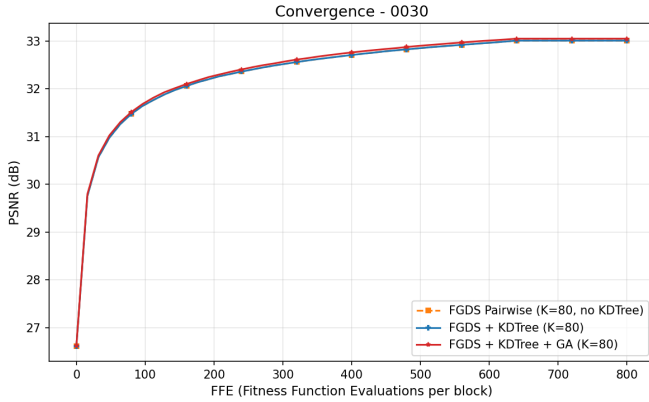


Fig. 2: E2 convergence on image 0030. Pairwise and KD-Tree coincide exactly; KD-Tree+GA is marginally higher. All saturate at $K \times 8 = 640$ FFEs.

C. E3: Choosing the pool size K

Table IV shows diminishing returns. PSNR rises steadily through $K=80$ (31.21 dB) but only +0.12 dB more at $K=160$, despite $\sim 1.24\times$ the encode time—evidence that the top-80 feature neighbors already contain already captures most high-quality candidates domain for each range block. We therefore adopt $K=80$ as the default sweet spot and use it in E2 and E4. Note that even $K=40$ (30.80 dB) already exceeds every PSO baseline in Table II at a fraction of their time, so the choice of K trades quality against speed within a regime that already dominates the metaheuristic methods.

TABLE IV: E3: K -sensitivity for KD-Tree FGDS, averaged over ten images. FFE = $K \times 8$.

K	FFE	PSNR (dB)	Avg MSE	Total (s)
5	40	29.18	92.23	51.6
10	80	29.80	79.34	95.7
20	160	30.33	69.68	183.6
40	320	30.80	62.10	360.9
80	640	31.21	56.27	712.9
160	1280	31.33	54.65	884.7

TABLE V: E4: feature-guided vs. random pool, both $K=80$ exhaustive, identical FFEs. PSNR (dB).

	0002	0007	0029	0030	0032
Random-80	27.97	26.00	22.75	29.60	32.78
FGDS-80	30.35	27.95	25.81	32.69	35.89
	0037	0043	0049	0051	0060
Random-80	31.16	26.96	29.01	27.15	26.21
FGDS-80	35.07	31.46	32.09	29.68	31.13

D. E4: Is it the features, or just a smaller pool?

A natural objection to FGDS is that any small exhaustive search might do well—perhaps the benefit is just the reduced space, not the feature guidance. E4 refutes this directly. We replace the feature-guided pool with a *random* pool of the same size ($K=80$) and run the identical exhaustive search; the only difference is how the 80 domains are chosen. As Table V and Fig. 3 show, FGDS beats the random pool on every image, by 1.95–4.92 dB (average +3.25 dB: 31.21 vs. 27.96 dB; Wilcoxon $p < 0.01$). A same-size random pool is therefore far from sufficient—the quality lives in the feature-guided *selection*. This is the empirical core of the paper’s thesis: the heuristic belongs in choosing which domains to consider.

E. Statistical significance

With $n=10$ paired images and FGDS winning every pair, each test attains $W^+ = 55$, $W^- = 0$, and exact one-sided $p = 1/2^{10} \approx 0.001$ (Table VI). Every claim in the paper—superiority over each PSO variant, the positive GA gain, and the advantage over a random pool—is statistically significant.

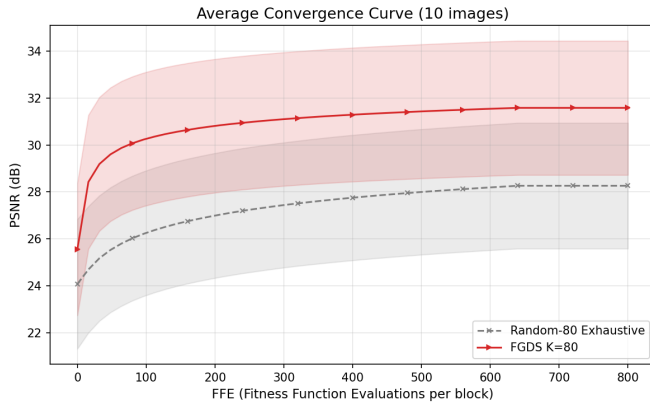


Fig. 3: E4 average convergence over ten images ($\pm 1\sigma$ bands). At equal FFEs the feature-guided pool (red) dominates a same-size random pool (gray) by ~ 3.25 dB throughout.

TABLE VI: Wilcoxon signed-rank tests (PSNR, one-sided, $n = 10$). W^+ is the positive-rank sum; all comparisons win on all ten images.

Comparison	Mean Δ PSNR	W^+	p
FGDS vs. PSO	+1.93	55	< 0.01
FGDS vs. Memetic PSO	+1.65	55	< 0.01
FGDS vs. FG-PSO	+0.53	55	< 0.01
FGDS+GA vs. FGDS	+0.05	55	< 0.01
FGDS-80 vs. Random-80	+3.25	55	< 0.01

F. Visual quality

Fig. 4 shows a representative reconstruction. At $CR \approx 3.7:1$ FGDS preserves the dominant structure—mountain ridges, snow boundaries, and smooth sky gradients—consistent with its 32.73 dB PSNR on this image.

VII. CONCLUSION

We set out to accelerate Fractal Image Compression with heuristics and, by following the evidence, arrived at a different place than the existing PSO-based literature. The lesson of our experiments is that for high-resolution FIC the limiting factor is not the optimizer but the search space it is given: a feature-guided pool of just 80 domains, searched exhaustively, beats PSO over half a million candidates by 1.9 dB, and once that pool exists the optimizer can be removed because exhaustive search dominates it. We therefore relocate the heuristic from block *matching* to block *selection*. The resulting method, FGDS, pairs an 8-dimensional isometry-invariant descriptor with KD-Tree nearest-neighbor selection—reducing pool construction $\sim 1390\times$ to under half a second—and a Genetic Algorithm that learns, once and offline, how much each feature should count, with weights that transfer across images because they reflect the structure of the affine model rather than image content. On ten 1024×1024 images FGDS improves PSNR by 1.7–3.2 dB over PSO and runs $\sim 4\times$ faster, all gains significant at $p < 0.01$, and a random-pool control attributes +3.25 dB specifically to the feature-guided selection.

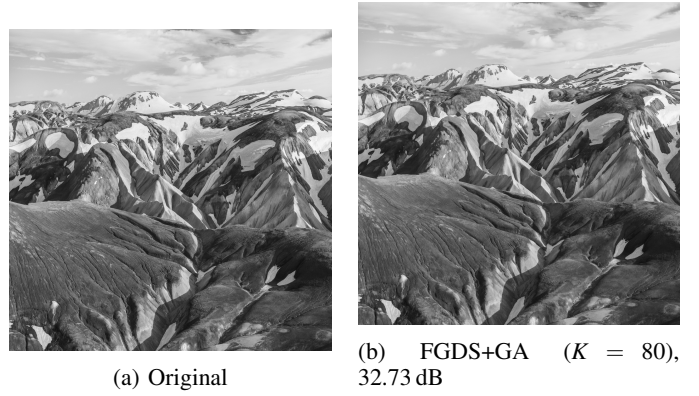


Fig. 4: Reconstruction on image 0030. FGDS preserves ridge structure, snow boundaries, and cloud gradients at $CR \approx 3.7:1$.

The broader contribution is the design principle: when a metaheuristic underperforms on a combinatorial problem, the remedy may be to apply it to the *construction of the search space* rather than to the search itself. The idea of learning which features matter for a selection task—rather than guessing them—is not specific to FIC and may transfer to other matching and retrieval problems. Future work includes color images, per-block adaptive K driven by block complexity, and replacing the hand-designed descriptor with a learned embedding.

ACKNOWLEDGMENT

The authors thank the Department of Computer Science and Information Engineering, National Taiwan Normal University.

REFERENCES

- [1] M. F. Barnsley and S. Demko, “Iterated function systems and the global construction of fractals,” *Proceedings of the Royal Society of London. A. Mathematical and Physical Sciences*, vol. 399, no. 1817, pp. 243–275, 06 1985. [Online]. Available: <https://doi.org/10.1098/rspa.1985.0057>
- [2] A. Jacquin, “Image coding based on a fractal theory of iterated contractive image transformations,” *IEEE Transactions on Image Processing*, vol. 1, no. 1, pp. 18–30, 1992.
- [3] A. Muruganandham and R. Wahida Banu, “Adaptive fractal image compression using pso,” *Procedia Computer Science*, vol. 2, pp. 338–344, 2010, proceedings of the International Conference and Exhibition on Biometrics Technology. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1877050910003741>
- [4] M.-S. Wu, J.-H. Jeng, and J.-G. Hsieh, “Schema genetic algorithm for fractal image compression,” *Engineering Applications of Artificial Intelligence*, vol. 20, no. 4, pp. 531–538, 2007.
- [5] M.-S. Wu, W.-C. Teng, J.-H. Jeng, and J.-G. Hsieh, “Spatial correlation genetic algorithm for fractal image compression,” *Chaos, Solitons & Fractals*, vol. 28, no. 2, pp. 497–510, 2006.
- [6] S. K. Mitra, C. Murthy, and M. K. Kundu, “Technique for fractal image compression using genetic algorithm,” *IEEE transactions on image processing*, vol. 7, no. 4, pp. 586–593, 1998.
- [7] T. Li, J. Shi, W. Deng, and Z. Hu, “Pyramid particle swarm optimization with novel strategies of competition and cooperation,” *Applied Soft Computing*, vol. 121, p. 108731, 2022.
- [8] P. Moscato, “On evolution, search, optimization, genetic algorithms and martial arts: Towards memetic algorithms,” California Institute of Technology, Tech. Rep. C3P Report 826, 1989.
- [9] Y. Fisher, Ed., *Fractal image compression: theory and application*. Berlin, Heidelberg: Springer-Verlag, 1995.

- [10] D. Saupe, "Accelerating fractal image compression by multi-dimensional nearest neighbor search," in *Proceedings DCC'95 Data Compression Conference*. IEEE, 1995, pp. 222–231.
- [11] M. F. Barnsley, *Fractals everywhere*. Academic press, 2014.
- [12] M. F. Barnsley and A. D. Sloan, "A better way to compress images," *Byte*, vol. 13, no. 1, pp. 215–223, 1988.
- [13] A. Jacquin, "Fractal image coding: a review," *Proceedings of the IEEE*, vol. 81, no. 10, pp. 1451–1465, 1993.
- [14] B. Bani-Eqbal, "Speeding up fractal image compression," in *IEE Colloquium on Fractals in Signal and Image Processing*, 1995, pp. 2/1–2/3.
- [15] B. Hürtgen and C. Stiller, "Fast hierarchical codebook search for fractal coding of still images," in *Video Communications and PACS for Medical Applications*, vol. 1977. SPIE, 1993, pp. 397–408.
- [16] C. Hufnagl and A. Uhl, "Algorithms for fractal image compression on massively parallel simd arrays," *Real-Time Imaging*, vol. 6, no. 4, pp. 267–281, 2000.
- [17] D. Vidya, R. Parthasarathy, T. Bina, and N. Swaroopa, "Architecture for fractal image compression," *Journal of systems architecture*, vol. 46, no. 14, pp. 1275–1291, 2000.
- [18] J. Kennedy and R. Eberhart, "Particle swarm optimization," in *Proceedings of ICNN'95-international conference on neural networks*, vol. 4. iee, 1995, pp. 1942–1948.
- [19] R. Eberhart and J. Kennedy, "A new optimizer using particle swarm theory," in *MHS'95. Proceedings of the sixth international symposium on micro machine and human science*. Ieee, 1995, pp. 39–43.
- [20] J. L. Bentley, "Multidimensional binary search trees used for associative searching," *Communications of the ACM*, vol. 18, no. 9, pp. 509–517, 1975.
- [21] J. H. Holland, *Adaptation in Natural and Artificial Systems*. Ann Arbor: University of Michigan Press, 1975.
- [22] L. J. Eshelman and J. D. Schaffer, "Real-coded genetic algorithms and interval-schemata," in *Foundations of Genetic Algorithms*, vol. 2. Elsevier, 1993, pp. 187–202.
- [23] E. Agustsson and R. Timofte, "Ntire 2017 challenge on single image super-resolution: Dataset and study," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops*, 2017, pp. 126–135.
- [24] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.